

Hospital Data Analysis using Hadoop MapReduce

Department-wise Patient Count (HospitalDepartmentCount job)

1. Objective

To process hospital patient records stored in HDFS with a Hadoop MapReduce program (HospitalDepartmentCount) that counts how many patients belong to each hospital department, and then to verify the result using HDFS and Linux commands (hdfs dfs, awk, sort, grep, head, tail).

- Keep the dataset hospital_data.csv in HDFS at /hospital/input.
- Run the MapReduce job and write the department-wise patient count to /hospital/output.
- Read the job log and counters to confirm the map and reduce phases ran correctly.
- Validate the output: highest and lowest counts, total, and filtered views.

2. Input

2.1 Dataset

The input is a CSV file stored in HDFS at /hospital/input/hospital_data.csv. It is 821 bytes and holds one header line plus 20 patient records (the job log shows Map input records = 21). Each record has six comma-separated fields:

Field	Description
Patient_ID	Unique patient identifier (P001, P002, ...)
Name	Patient name
Age	Age in years
Department	Hospital department - the field used as the grouping key
Diagnosis	Medical condition
Admission_Days	Number of days admitted

Records visible in the terminal screenshots (P001 to P019):

Patient_ID	Name	Age	Department	Diagnosis	Days
P001	Ravi	45	Cardiology	Heart_Disease	5
P002	Anu	32	General_Medicine	Fever	2
P003	Kiran	60	Orthopedics	Fracture	7
P004	Meena	28	Pediatrics	Infection	3
P005	Arjun	51	Neurology	Migraine	4
P006	Priya	39	Dermatology	Skin_Allergy	2
P007	Suresh	67	Cardiology	Hypertension	6
P008	Divya	25	General_Medicine	Fever	2
P009	Manoj	55	Orthopedics	Arthritis	5
P010	Lakshmi	10	Pediatrics	Infection	3
P011	Rahul	48	Neurology	Epilepsy	8
P012	Shalini	35	Dermatology	Eczema	4

Patient_ID	Name	Age	Department	Diagnosis	Days
P013	Vijay	42	Cardiology	Heart_Disease	6
P014	Nisha	29	General_Medicine	Diabetes	5
P015	Anil	63	Orthopedics	Fracture	9
P016	Swathi	8	Pediatrics	Fever	2
P017	Ramesh	57	Neurology	Migraine	3
P018	Kavya	31	Dermatology	Skin_Allergy	2
P019	Deepak	50	Cardiology	Hypertension	5

Note: the 20th record is not visible in the screenshots because the awk filter used there prints at most 3 records per diagnosis.

2.2 Commands and explanation

Command	Explanation
<code>stop-yarn.sh && sleep 3 && start-yarn.sh</code>	Restarts YARN so the new configuration is loaded.
<code>jps</code>	Lists running Hadoop daemons to confirm the cluster is up.
<code>hadoop jar hospital-mapreduce.jar HospitalDepartmentCount /hospital/input/hospital_data.csv /hospital/output</code>	Runs the job. Arguments: the main class, the HDFS input file and the HDFS output directory (which must not already exist).
<code>hdfs dfs -cat /hospital/output/part-r-00000</code>	Prints the reducer output file from HDFS.
<code>sort, awk, grep, head, tail (piped)</code>	Linux tools used to analyse the output and to check the input data.

2.3 Program logic

The program source is not part of the screenshots, so the logic below is inferred from the job counters and the output:

- Mapper: reads each line, skips the header, takes the Department (4th field) and emits (Department, 1). Map input records = 21 but map output records = 20, which shows the header line was skipped.
- Shuffle and sort: groups the 20 pairs by department into 6 groups (Reduce input groups = 6).
- Reducer: adds up the 1s for each department and writes (Department, count). Reduce output records = 6.

3. Output (screenshots with explanation)

Step 1: YARN configuration and restart

```
Menu 1:15 PM
aiml@aiml-D206-77: ~
File Edit View Search Terminal Help
2026-10-01 12:19:56,417 INFO mapreduce.Job: Counters: 0
aiml@aiml-D206-77:~$ echo 'yarn.nodemanager.delete.debug-delay-sec=600' >> $HADOOP_HOME/etc/hadoop/yarn-site.xml
aiml@aiml-D206-77:~$ sed -i '/yarn.nodemanager.delete.debug-delay-sec=600/d' $HADOOP_HOME/etc/hadoop/yarn-site.xml
aiml@aiml-D206-77:~$ sed -i '/</configuration>/i\ <property>\n <name>yarn.nodemanager.delete.debug-delay-sec</name>\n <value>600</value>\n </property>' $HADOOP_HOME/etc/hadoop/yarn-site.xml
aiml@aiml-D206-77:~$ tail -20 $HADOOP_HOME/etc/hadoop/yarn-site.xml
<property>
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
</property>

<property>
  <name>yarn.nodemanager.aux-services.mapreduce_shuffle.class</name>
  <value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>

<property>
  <name>yarn.nodemanager.env-whitelist</name>
  <value>JAVA_HOME,HADOOP_HOME,HADOOP_CONF_DIR,HADOOP_MAPRED_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_YARN_HOME</value>
</property>

<property>
  <name>yarn.nodemanager.delete.debug-delay-sec</name>
  <value>600</value>
</property>
</configuration>
aiml@aiml-D206-77:~$ stop-yarn.sh && sleep 3 && start-yarn.sh
Stopping nodemanagers
localhost: WARNING: nodemanager did not stop gracefully after 5 seconds: Trying to kill with kill -9
Stopping resourcemanager
Starting resourcemanager
Starting nodemanagers
aiml@aiml-D206-77:~$ jps
3761 NameNode
10882 Jps
4213 SecondaryNameNode
10229 ResourceManager
10602 NodeManager
3967 DataNode
```

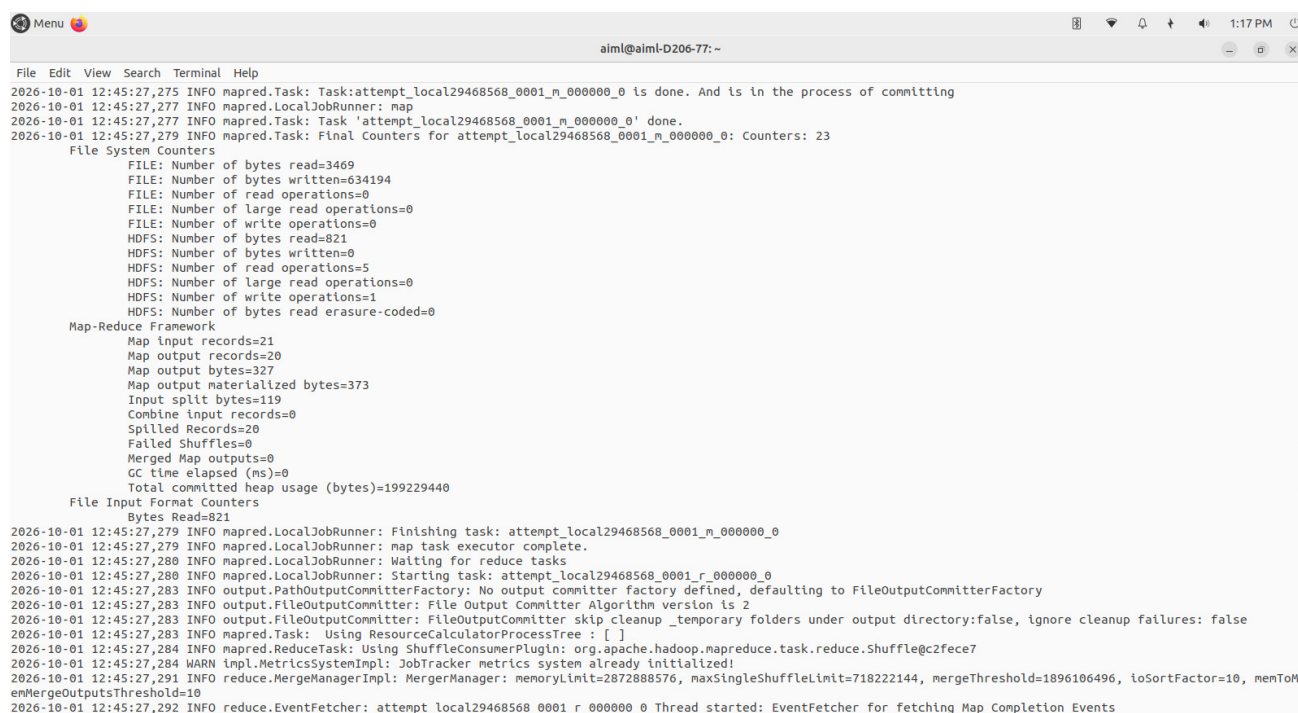
The property `yarn.nodemanager.delete.debug-delay-sec=600` was added to `yarn-site.xml`, which keeps container files and logs for 600 seconds after an application finishes. The `echo` command appended it after the closing tag, so `sed` was used to remove that line and insert the property correctly before `</configuration>`. `tail -20` confirms the property is now inside the configuration block. YARN was then restarted with `stop-yarn.sh && start-yarn.sh` (the NodeManager needed `kill -9` to stop). `jps` then lists NameNode, SecondaryNameNode, DataNode, ResourceManager and NodeManager, so all daemons are running.

Step 2: Submitting the MapReduce job

```
Menu 1:16 PM
aiml@aiml-D206-77: ~
File Edit View Search Terminal Help
Can not find any log file matching the pattern: [ALL] for the application: application_1790838544534_0001
Can not find the logs for the application: application_1790838544534_0001 with the appOwner: aiml
aiml@aiml-D206-77:~$ nano /usr/local/hadoop/etc/hadoop/mapred-site.xml
aiml@aiml-D206-77:~$ hadoop jar hospital-mapreduce.jar HospitalDepartmentCount /hospital/input/hospital_data.csv /hospital/output
2026-10-01 12:45:26,893 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-10-01 12:45:26,929 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-10-01 12:45:26,929 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-10-01 12:45:26,980 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-10-01 12:45:27,020 INFO input.FileInputFormat: Total input files to process : 1
2026-10-01 12:45:27,046 INFO mapreduce.JobSubmitter: number of splits:1
2026-10-01 12:45:27,100 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local29468568_0001
2026-10-01 12:45:27,100 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-10-01 12:45:27,163 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-10-01 12:45:27,163 INFO mapreduce.Job: Running job: job_local29468568_0001
2026-10-01 12:45:27,164 INFO mapred.LocalJobRunner: OutputCommiter set in config null
2026-10-01 12:45:27,166 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-10-01 12:45:27,166 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-10-01 12:45:27,166 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup_temporary folders under output directory:false, ignore cleanup failures: false
2026-10-01 12:45:27,167 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
2026-10-01 12:45:27,187 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-10-01 12:45:27,187 INFO mapred.LocalJobRunner: Starting task: attempt_local29468568_0001_m_000000_0
2026-10-01 12:45:27,196 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-10-01 12:45:27,196 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-10-01 12:45:27,196 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup_temporary folders under output directory:false, ignore cleanup failures: false
2026-10-01 12:45:27,202 INFO mapred.Task: Using ResourceCalculatorProcessTree : [ ]
2026-10-01 12:45:27,204 INFO mapred.MapTask: Processing split: hdfs://localhost:9000/hospital/input/hospital_data.csv:0+821
2026-10-01 12:45:27,239 INFO mapred.MapTask: (EQUATOR) 0 kvi 26214396(104857584)
2026-10-01 12:45:27,239 INFO mapred.MapTask: mapreduce.task.io.sort.mb: 100
2026-10-01 12:45:27,239 INFO mapred.MapTask: soft limit at 83886080
2026-10-01 12:45:27,239 INFO mapred.MapTask: bufstart = 0; bufvoid = 104857600
2026-10-01 12:45:27,239 INFO mapred.MapTask: kvstart = 26214396; length = 6553600
2026-10-01 12:45:27,241 INFO mapred.MapTask: Map output collector class = org.apache.hadoop.mapred.MapTask$MapOutputBuffer
2026-10-01 12:45:27,267 INFO mapred.LocalJobRunner:
2026-10-01 12:45:27,268 INFO mapred.MapTask: Starting flush of map output
2026-10-01 12:45:27,268 INFO mapred.MapTask: Spilling map output
2026-10-01 12:45:27,268 INFO mapred.MapTask: bufstart = 0; bufend = 327; bufvoid = 104857600
2026-10-01 12:45:27,268 INFO mapred.MapTask: kvstart = 26214396(104857584); kvoid = 26214320(104857280); length = 77/6553600
2026-10-01 12:45:27,271 INFO mapred.MapTask: Finished split 0
2026-10-01 12:45:27,275 INFO mapred.Task: Task:attempt_local29468568_0001_m_000000_0 is done. And is in the process of committing
2026-10-01 12:45:27,277 INFO mapred.LocalJobRunner: map
2026-10-01 12:45:27,277 INFO mapred.Task: Task 'attempt_local29468568_0001_m_000000_0' done.
```

The job is launched with `hadoop jar hospital-mapreduce.jar HospitalDepartmentCount /hospital/input/hospital_data.csv /hospital/output`. The log shows Total input files to process: 1 and number of splits: 1, so one map task is created. The job id is `job_local29468568_0001` and it runs through LocalJobRunner (local mode). The map task processes the split `hdfs://localhost:9000/hospital/input/hospital_data.csv:0+821`, meaning bytes 0 to 821, the whole file. The map output is then spilled (327 bytes). The warning about the Tool interface is harmless. At the top, an earlier 'Can not find any log file' message from a previous YARN application is also visible.

Step 3: Map task completed



```
2026-10-01 12:45:27,275 INFO mapred.Task: Task:attempt_local29468568_0001_m_000000_0 is done. And is in the process of committing
2026-10-01 12:45:27,277 INFO mapred.LocalJobRunner: map
2026-10-01 12:45:27,277 INFO mapred.Task: Task 'attempt_local29468568_0001_m_000000_0' done.
2026-10-01 12:45:27,279 INFO mapred.Task: Final Counters for attempt_local29468568_0001_m_000000_0: Counters: 23
  File System Counters
    FILE: Number of bytes read=3469
    FILE: Number of bytes written=634194
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=821
    HDFS: Number of bytes written=0
    HDFS: Number of read operations=5
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=1
    HDFS: Number of bytes read erasure-coded=0
  Map-Reduce Framework
    Map input records=21
    Map output records=20
    Map output bytes=327
    Map output materialized bytes=373
    Input split bytes=119
    Combine input records=0
    Spilled Records=20
    Failed Shuffles=0
    Merged Map outputs=0
    GC time elapsed (ms)=0
    Total committed heap usage (bytes)=199229440
  File Input Format Counters
    Bytes Read=821
2026-10-01 12:45:27,279 INFO mapred.LocalJobRunner: Finishing task: attempt_local29468568_0001_m_000000_0
2026-10-01 12:45:27,279 INFO mapred.LocalJobRunner: map task executor complete.
2026-10-01 12:45:27,280 INFO mapred.LocalJobRunner: Waiting for reduce tasks
2026-10-01 12:45:27,280 INFO mapred.LocalJobRunner: Starting task: attempt_local29468568_0001_r_000000_0
2026-10-01 12:45:27,283 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-10-01 12:45:27,283 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-10-01 12:45:27,283 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup_temporary folders under output directory:false, ignore cleanup failures: false
2026-10-01 12:45:27,283 INFO mapred.Task: Using ResourceCalculatorProcessTree : [ ]
2026-10-01 12:45:27,284 INFO mapred.ReduceTask: Using ShuffleConsumerPlugin: org.apache.hadoop.mapreduce.task.reduce.Shuffle@c2fece7
2026-10-01 12:45:27,284 WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
2026-10-01 12:45:27,291 INFO reduce.MergeManagerImpl: MergerManager: memoryLimit=2872888576, maxSingleShuffleLimit=718222144, mergeThreshold=1896106496, ioSortFactor=10, memToMergeOutputsThreshold=10
2026-10-01 12:45:27,292 INFO reduce.EventFetcher: attempt_local29468568_0001_r_000000_0 Thread started: EventFetcher for fetching Map Completion Events
```

The map task finishes. Its counters show Map input records = 21 (header + 20 records) and Map output records = 20, so the header was ignored. Map output bytes = 327, HDFS bytes read = 821 (the file size) and Input split bytes = 119. The log then shows the reduce task starting and the shuffle fetcher collecting map output.

Step 4: Reduce task completed

```
Menu
aiml@aiml-D206-77: ~
file Edit View Search Terminal Help
926-10-01 12:45:27,352 INFO mapred.Task: Task attempt_local29468568_0001_r_000000_0 is allowed to commit now
926-10-01 12:45:27,359 INFO output.FileOutputCommitter: Saved output of task 'attempt_local29468568_0001_r_000000_0' to hdfs://localhost:9000/hospital/output
926-10-01 12:45:27,359 INFO mapred.LocalJobRunner: reduce > reduce
926-10-01 12:45:27,359 INFO mapred.Task: Task 'attempt_local29468568_0001_r_000000_0' done.
926-10-01 12:45:27,360 INFO mapred.Task: Final Counters For attempt_local29468568_0001_r_000000_0: Counters: 30
File System Counters
  FILE: Number of bytes read=4247
  FILE: Number of bytes written=634567
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=821
  HDFS: Number of bytes written=85
  HDFS: Number of read operations=10
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=3
  HDFS: Number of bytes read erasure-coded=0
Map-Reduce Framework
  Combine input records=0
  Combine output records=0
  Reduce input groups=6
  Reduce shuffle bytes=373
  Reduce input records=20
  Reduce output records=6
  Spilled Records=20
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=6
  Total committed heap usage (bytes)=239075328
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Output Format Counters
  Bytes Written=85
926-10-01 12:45:27,360 INFO mapred.LocalJobRunner: Finishing task: attempt_local29468568_0001_r_000000_0
926-10-01 12:45:27,360 INFO mapred.LocalJobRunner: reduce task executor complete.
926-10-01 12:45:28,166 INFO mapreduce.Job: Job job_local29468568_0001 running in uber mode : false
926-10-01 12:45:28,167 INFO mapreduce.Job: map 100% reduce 100%
926-10-01 12:45:28,167 INFO mapreduce.Job: Job job_local29468568_0001 completed successfully
```

The reduce task commits its output to hdfs://localhost:9000/hospital/output. Reduce input groups = 6 (one per department), Reduce input records = 20, Reduce output records = 6 and HDFS bytes written = 85. All shuffle error counters (BAD_ID, CONNECTION, IO_ERROR, WRONG_LENGTH, WRONG_MAP, WRONG_REDUCE) are 0. The log ends with map 100% reduce 100%.

Step 5: Job completed successfully

```
Bytes Written=85
2026-10-01 12:45:27,360 INFO mapred.LocalJobRunner: Finishing task: attempt_local29468568_0001_r_000000_0
2026-10-01 12:45:27,360 INFO mapred.LocalJobRunner: reduce task executor complete.
2026-10-01 12:45:28,166 INFO mapreduce.Job: Job job_local29468568_0001 running in uber mode : false
2026-10-01 12:45:28,167 INFO mapreduce.Job: map 100% reduce 100%
2026-10-01 12:45:28,169 INFO mapreduce.Job: Job job_local29468568_0001 completed successfully
2026-10-01 12:45:28,177 INFO mapreduce.Job: Counters: 36
File System Counters
  FILE: Number of bytes read=7716
  FILE: Number of bytes written=1268761
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=1642
  HDFS: Number of bytes written=85
  HDFS: Number of read operations=15
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=4
  HDFS: Number of bytes read erasure-coded=0
Map-Reduce Framework
  Map input records=21
  Map output records=20
  Map output bytes=327
  Map output materialized bytes=373
  Input split bytes=119
  Combine input records=0
  Combine output records=0
  Reduce input groups=6
  Reduce shuffle bytes=373
  Reduce input records=20
  Reduce output records=6
  Spilled Records=40
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=6
  Total committed heap usage (bytes)=438304768
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
```

The log confirms that the job completed successfully with 36 counters. The job-level totals match the task-level values: Map input records = 21, Map output records = 20, Reduce input groups = 6, Reduce input records = 20, Reduce output records = 6, Shuffled Maps = 1, Failed Shuffles = 0 and HDFS bytes written = 85.

Step 6: Final output and analysis

```
Menu
aiml@aiml-D206-77: ~
File Edit View Search Terminal Help
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=821
File Output Format Counters
  Bytes Written=85
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/output/part-r-00000
Cardiology 4
Dermatology 3
General_Medicine 4
Neurology 3
Orthopedics 3
Pediatrics 3
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/output/part-r-00000 | sort -k2,2nr | head -1
Cardiology 4
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/output/part-r-00000 | sort -k2,2n | head -1
Dermatology 3
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/output/part-r-00000 | awk '{sum += $2} END {print sum}'
20
aiml@aiml-D206-77:~$
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/output/part-r-00000 | awk '$2 > 3'
Cardiology 4
General_Medicine 4
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/output/part-r-00000 | awk '$2 == 3'
Command 'dfs' not found, did you mean:
  command 'dss' from snap data-science-stack (0.1-0fdd0b77cd50bb922f643b17899b)
  command 'dfc' from deb dfc (3.1.1-1)
  command 'dfs' from deb openafs-client (1.8.10-2ubuntu1-22.04.2)
  command 'dcs' from deb drbl (4.5.16-1)
  command 'zfs' from deb zfsutils-linux (2.1.5-1ubuntu6-22.04.6)
  command 'zfs' from deb zfs-fuse (0.7.0-22ubuntu1)
  command 'bfs' from deb bfs (2.3.1-1)
  command 'dns' from deb anacrolx-dns (1.1.0-2ubuntu0.1)
  command 'df' from deb coreutils (8.32-4.1ubuntu1.3)
  command 'hfs' from deb hfsutils-tcltk (3.2.6-15build2)
  command 'dfrs' from deb dfrs (0.0.7-2ubuntu0.1)
See 'snap info <snapname>' for additional versions.
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/output/part-r-00000 | awk '{sum += $2} END {print sum}'
20
aiml@aiml-D206-77:~$
```

`hdfs dfs -cat /hospital/output/part-r-00000` shows the six department counts: Cardiology 4, Dermatology 3, General_Medicine 4, Neurology 3, Orthopedics 3, Pediatrics 3. Follow-up commands: `sort -k2,2nr | head -1` gives the top department (Cardiology 4, tied with General_Medicine); `sort -k2,2n | head -1` gives the lowest (Dermatology 3, tied with three others); `awk '{sum += $2} END {print sum}'` prints 20, equal to the number of records; `awk '$2 > 3'` lists Cardiology and General_Medicine. The command typed as `'dfs dfs -cat ...'` failed with 'Command dfs not found' because the leading `'hdfs'` was missing. The top of the screenshot also shows the final counters (Bytes Read = 821, Bytes Written = 85, no shuffle errors).

Step 7: Filtering the output and checking the input

```
Menu
aiml@aiml-D206-77: ~
File Edit View Search Terminal Help
  command 'hfs' from deb hfsutils-tcltk (3.2.6-15build2)
  command 'dfrs' from deb dfrs (0.0.7-2ubuntu0.1)
See 'snap info <snapname>' for additional versions.
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/output/part-r-00000 | awk '$2 == 3'
Dermatology 3
Neurology 3
Orthopedics 3
Pediatrics 3
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | grep "Cardiology" | head -3
P001,Ravi,45,Cardiology,Heart_Disease,5
P007,Suresh,67,Cardiology,Hypertension,6
P013,Vijay,42,Cardiology,Heart_Disease,6
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' 'NR>1 {count[$5]++; if(count[$5]<=3) print}'
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
P004,Meena,28,Pediatrics,Infection,3
P005,Arjun,51,Neurology,Migraine,4
P006,Priya,39,Dermatology,Skin_Allergy,2
P007,Suresh,67,Cardiology,Hypertension,6
P008,Divya,25,General_Medicine,Fever,2
P009,Manoj,55,Orthopedics,Arthritis,5
P010,Lakshmi,10,Pediatrics,Infection,3
P011,Rahul,48,Neurology,Epilepsy,8
P012,Shalini,35,Dermatology,Eczema,4
P013,Vijay,42,Cardiology,Heart_Disease,6
P014,Nisha,29,General_Medicine,Diabetes,5
P015,Anil,63,Orthopedics,Fracture,9
P016,Swathi,8,Pediatrics,Fever,2
P017,Ramesh,57,Neurology,Migraine,3
P018,Kavya,31,Dermatology,Skin_Allergy,2
P019,Deepak,50,Cardiology,Hypertension,5
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | tail -n +2 | head -3
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | grep "^15,"
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' 'NR>1 {count[$5]++; if(count[$5]<=3) print}'
aiml@aiml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | head -5
Patient_ID,Name,Age,Department,Diagnosis,Admission_Days
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
P004,Meena,28,Pediatrics,Infection,3
```

awk '\$2 == 3' lists the four departments with exactly 3 patients: Dermatology, Neurology, Orthopedics and Pediatrics. Next, the input file is checked directly: grep "Cardiology" | head -3 returns P001 Ravi, P007 Suresh and P013 Vijay (the first three of the four Cardiology patients). The awk command 'NR>1 {count[\$5]++; if(count[\$5]<=3) print}' skips the header and prints at most three records per diagnosis (field 5), which is why only P001 to P019 are shown. tail -n +2 | head -3 skips the header and shows the first three records. grep "^15," and awk '\$1=="15"' return nothing because patient IDs are written as P015, not 15.

Step 8: Looking up a single patient

```
P001,Ravi,45,Cardiology,Heart_Disease,5
P007,Suresh,67,Cardiology,Hypertension,6
P013,Vijay,42,Cardiology,Heart_Disease,6
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' 'NR>1 {count[$5]++; if(count[$5]<=3) print}'
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
P004,Meena,28,Pediatrics,Infection,3
P005,Arjun,51,Neurology,Migraine,4
P006,Priya,39,Dermatology,Skin_Allergy,2
P007,Suresh,67,Cardiology,Hypertension,6
P008,Divya,25,General_Medicine,Fever,2
P009,Manoj,55,Orthopedics,Arthritis,5
P010,Lakshmi,10,Pediatrics,Infection,3
P011,Rahul,48,Neurology,Epilepsy,8
P012,Shalini,35,Dermatology,Eczema,4
P013,Vijay,42,Cardiology,Heart_Disease,6
P014,Nisha,29,General_Medicine,Diabetes,5
P015,Anil,63,Orthopedics,Fracture,9
P016,Swathi,8,Pediatrics,Fever,2
P017,Ramesh,57,Neurology,Migraine,3
P018,Kavya,31,Dermatology,Skin_Allergy,2
P019,Deepak,50,Cardiology,Hypertension,5
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | tail -n +2 | head -3
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | grep "^15,"
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $1=="15" '
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | head -5
Patient_ID,Name,Age,Department,Diagnosis,Admission_Days
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
P004,Meena,28,Pediatrics,Infection,3
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $1=="15" '
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $1=="P015" {print "Patient ID:", $1, "Department:", $4}'
Patient ID: P015 Department: Orthopedics
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $4=="Cardiology" ' | head -3
P001,Ravi,45,Cardiology,Heart_Disease,5
P007,Suresh,67,Cardiology,Hypertension,6
P013,Vijay,42,Cardiology,Heart_Disease,6
ainl@auml-D206-77:~$ █
```

This view repeats the previous checks and then continues. head -5 shows the header and the first four records. After the two empty searches for ID 15, the corrected command awk -F',' '\$1=="P015" {print "Patient ID:", \$1, "Department:", \$4}' prints 'Patient ID: P015 Department: Orthopedics'. Finally awk -F',' '\$4=="Cardiology" | head -3 prints the first three Cardiology records (P001, P007, P013). -F',' tells awk to split fields on commas.

Step 9: Final terminal view

```
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' 'NR>1 {count[$5]++; if(count[$5]<=3) print}'
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
P004,Meena,28,Pediatrics,Infection,3
P005,Arjun,51,Neurology,Migraine,4
P006,Priya,39,Dermatology,Skin_Allergy,2
P007,Suresh,67,Cardiology,Hypertension,6
P008,Divya,25,General_Medicine,Fever,2
P009,Manoj,55,Orthopedics,Arthritis,5
P010,Lakshmi,10,Pediatrics,Infection,3
P011,Rahul,48,Neurology,Epilepsy,8
P012,Shalini,35,Dermatology,Eczema,4
P013,Vijay,42,Cardiology,Heart_Disease,6
P014,Nisha,29,General_Medicine,Diabetes,5
P015,Anil,63,Orthopedics,Fracture,9
P016,Swathi,8,Pediatrics,Fever,2
P017,Ramesh,57,Neurology,Migraine,3
P018,Kavya,31,Dermatology,Skin_Allergy,2
P019,Deepak,50,Cardiology,Hypertension,5
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | tail -n +2 | head -3
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | grep "^15,"
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $1=="15" '
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | head -5
Patient_ID,Name,Age,Department,Diagnosis,Admission_Days
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
P004,Meena,28,Pediatrics,Infection,3
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $1=="15" '
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $1=="P015" {print "Patient ID:", $1, "Department:", $4}'
Patient ID: P015 Department: Orthopedics
ainl@auml-D206-77:~$ hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $4=="Cardiology" ' | head -3
P001,Ravi,45,Cardiology,Heart_Disease,5
P007,Suresh,67,Cardiology,Hypertension,6
P013,Vijay,42,Cardiology,Heart_Disease,6
ainl@auml-D206-77:~$ █
```

A later scroll position of the same terminal session. It shows the last commands again: the awk filters, the P015 lookup and the Cardiology filter. It confirms the data checks end with the expected results and that the input records match the MapReduce output.

Final output of the job

Department	Patient count
Cardiology	4
Dermatology	3
General_Medicine	4
Neurology	3
Orthopedics	3
Pediatrics	3
Total	20

4. Outcome

- The HospitalDepartmentCount job completed successfully on 20 patient records and produced 6 department counts, stored in /hospital/output/part-r-00000.
- Cardiology and General_Medicine have the most patients (4 each). Dermatology, Neurology, Orthopedics and Pediatrics have 3 each.
- The counts add up to 20, which matches the number of records in the input file, so the result is verified.
- The counters are consistent: 20 map output records = 20 reduce input records, and 6 reduce groups = 6 output lines. There were no failed shuffles or shuffle errors.
- HDFS commands combined with awk, sort and grep give a quick way to validate MapReduce output and inspect the source data.
- Practical lessons: the output directory must not exist before a run, patient IDs include the 'P' prefix, and the command must start with `hdfs dfs` (not `dfs dfs`).

Conclusion: Hadoop MapReduce successfully aggregated the hospital dataset by department, and the output was cross-checked against the input data.